

# OOPS CONCEPT IN PYTHON

# What is OOP?

Before **OOP**, code was written as a long list of instructions. As projects grew bigger, it became messy and hard to manage. OOP solves this by organizing code around real-world objects.

- Everything in the real world is an object – OOP brings that thinking into code
- Code is organized into classes, making it clean and structured
- Easier to debug, extend, and maintain large applications
- Built on 4 core pillars: **Encapsulation, Inheritance, Polymorphism, Abstraction**

# Core OOP concept

- Classes
- Objects
- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

# Classes & Objects

A **class** is just a template. It doesn't hold any real data on its own. When you create an **object** from a class, that's when actual data gets stored and actions can be performed.

- **Class** = Blueprint (e.g. design plan of a house)
- **Object** = Actual instance built from that blueprint (e.g. the real house)
- A class defines what **attributes** (data) and **methods** (actions) an object will have
- Multiple objects can be created from the same class, each with their own data
- **Example:** "Car" is a class. Your Honda City and your friend's Swift are two different objects

# Inheritance

Inheritance allows one class to acquire the properties and methods of another class. This avoids writing the same code again and again, and makes your codebase much cleaner.

- **Parent class** (also called **base** class) contains common, shared features
- **Child class** (also called **derived** class) inherits everything from the parent
- Child class can also add its own new attributes and methods on top
- Changes made in the parent class automatically reflect in child classes
- Real life example: **Animal** class has **eat()** and **sleep()**. Dog and Cat inherit these, and Dog adds **bark()**, Cat adds **meow()**



# Polymorphism

Poly means **many**, morph means **forms**. Polymorphism allows the same method name to behave differently depending on which object calls it. This makes your code highly flexible and extensible.

- Same method name works **differently** for different classes
- You don't need to remember multiple method names for similar actions
- **Method Overriding**: Child class provides its own version of a parent's method

Real life example: A "**draw()**" method on a Circle draws a **circle**, on a Rectangle it draws a **rectangle** — same name, different behavior

# Encapsulation

Encapsulation is about protecting the internal data of a class. You control what the outside world can see and what it cannot. This prevents accidental modification of important data.

- Data and methods that operate on that data are bundled together inside a class
- Sensitive data is marked private so it can't be accessed directly from outside
- Access is given through controlled methods called **getters** and **setters**
- Makes code more secure and predictable
- **Real life example:** ATM machine you can check balance and withdraw, but you can't access the bank's internal system directly

# Abstraction

Abstraction is about reducing complexity. You expose only the necessary details to the user and hide all the complicated internal logic. The user doesn't need to know how things work inside, just that they work.